Pip: An Offline Generative AI Voice Companion via QLoRA Fine-Tuning of a 1.5B Small Language Model

Real-Time On-Device Roleplay, Dynamic 8-Bit Graph Lowering, Cascaded Neural Speech, and 60 Hz Procedural Animation

Author & Project Details:

Shambhu Shivanand Khanai
Third Year Undergraduate, Department of Statistics and Data Science
Indian Institute of Technology Kanpur (IITK)

Course: CS787 — Generative Artificial Intelligence
Target Platform: Android Edge Runtime (`arm64-v8a`, Tested on Snapdragon 8 Gen 2 Tablet / 6 GB Phone Target)

Project Repository: https://github.com/Shambhu2025/Pip-GenAI-Project

November 13, 2026

Contents

1 Introduction **2**
- 1.1 Motivation for Offline Embodied Voice Companions . . . 2
- 1.2 Limitations of Cloud-Centric Conversational Pipelines . . . 2
- 1.3 Need for Parameter-Efficient On-Device Adaptation . . . 2

2 Background and Theoretical Foundations **3**
- 2.1 Autoregressive Decoder-Only Small Language Models . . . 3
- 2.2 Block-Wise k-Bit Quantization, NormalFloat4, and Double Quantization . . . 3
- 2.3 Low-Rank Adaptation (LoRA) and QLoRA Backpropagation . . . 4
- 2.4 Neural Speech Perception and Generative VITS Synthesis . . . 5

3 Problem Statement **5**
- 3.1 Challenges in Real-Time On-Device Roleplay Under Mobile Memory Ceilings . . . 5
- 3.2 Limitations Observed Using the Untuned `Qwen3-0.6B` Baseline . . . 6
- 3.3 Hidden Reasoning Token Overhead and First-Token Latency Degradation . . . 6
- 3.4 Persona Collapse, Formatting Violations, and STT Homophone Drift . . . 6

4 System Architecture and Generative AI Implementation **7**
- 4.1 Proposed Cascaded Three-Stage On-Device Neural Architecture . . . 7
- 4.2 Multi-Scenario Dataset Construction and ChatML Pre-Processing . . . 8
- 4.3 QLoRA Fine-Tuning Strategy Across All Seven Linear Projections . . . 9
- 4.4 PyTorch-to-LiteRT-LM Graph Lowering and `dynamic_wi8_afp32` Quantization . . . 11
- 4.5 Embodied 60 Hz Procedural Animation, RMS Lip-Sync, and Idle Scheduling . . . 12
- 4.6 Evaluation Procedure and Five-Dimensional Rubric Formulation . . . 13

5 Results and Discussion **14**
- 5.1 Performance Comparison: Untuned Baseline vs. QLoRA Fine-Tuned Model . . . 14
- 5.2 QLoRA Training Convergence and On-Device Quantization Behavior . . . 15
- 5.3 On-Device Latency Breakdown, Memory Footprint, and Precision Telemetry . . . 16
- 5.4 Live 21-Turn Evaluation Log: OOD Generalization, Error Audit, and Memorization Analysis . . . 18

6 Conclusion **20**
- 6.1 Summary of Findings . . . 20
- 6.2 Effectiveness of the QLoRA and LiteRT-LM Edge Architecture . . . 21
- 6.3 Practical Implications for Privacy-Preserving Mobile AI . . . 21

7 Design Alternatives Evaluated, Limitations, and Future Work **21**
- 7.1 Design Alternatives Considered: Empirical Tests vs. Literature Constraints . . . 21
- 7.2 System Limitations Observed in Deployment . . . 22
- 7.3 Future Work 1: Grammar-Mask the First-Token Control-Tag Branch . . . 23
- 7.4 Future Work 2: Cross-Turn Episodic Memory via On-Device Vector Retrieval . . . 23
- 7.5 Future Work 3: Hexagon NPU Delegation and Emotion-Conditioned VITS . . . 23

1 Introduction

1.1 Motivation for Offline Embodied Voice Companions

Spoken human-computer interaction represents the most natural interface for conversational companions, educational peer tutors, and interactive roleplay agents. However, transforming a raw pretrained language model into an expressive, real-time voice companion requires solving two coupled generative problems simultaneously: synthesizing contextually faithful, persona-consistent spoken dialogue, and driving synchronized visual embodiment (facial expressions, lip movements, and physical body kinematics) within a conversational turnaround window of under two seconds. When deployed directly on mobile hardware without internet connectivity, such an agent guarantees complete user privacy, zero recurring server inference costs, and deterministic availability across offline environments.

1.2 Limitations of Cloud-Centric Conversational Pipelines

Conventional voice assistants rely on a three-tier cloud pipeline in which recorded microphone audio is streamed over a network to remote speech-to-text (STT) servers, processed by a datacenter Large Language Model (LLM), and returned via cloud text-to-speech (TTS) synthesis. This architecture suffers from three structural bottlenecks:

- **Network Latency and Jitter:** Round-trip network transport across cellular or congested Wi-Fi links introduces variable delays of 800–2,500 ms before audio playback begins, breaking conversational immersion.
- **Privacy Exposure:** Continuous transmission of indoor voice recordings and personal conversations to third-party endpoints creates severe privacy and regulatory liabilities.
- **API Cost Scaling:** Continuous full-duplex voice interaction with proactive idle check-ins rapidly accumulates token and audio synthesis charges that are unsustainable for consumer mobile applications.

1.3 Need for Parameter-Efficient On-Device Adaptation

Deploying a general-purpose Small Language Model (SLM) directly onto a mobile System-on-Chip (SoC) does not automatically yield a usable voice companion [7]. Out-of-the-box instruction-tuned models exhibit verbose textbook formatting, emit markdown bullet lists and Unicode emojis that corrupt downstream TTS engines, break character during imaginative roleplay, and fail to emit structured kinematic control tokens required by procedural animation engines. Full 16-bit parameter fine-tuning of billion-parameter models is computationally prohibitive on single-GPU academic hardware and risks catastrophic forgetting of general world knowledge. Consequently, there is a critical need to combine Parameter-Efficient Fine-Tuning via Quantized Low-Rank Adaptation (**QLoRA**) [1], post-training mobile graph lowering (**LiteRT-LM**) [8], and offline neural speech models (**Whisper** [6] and **Piper VITS** [5]) into a unified, memory-bounded on-device runtime. The complete Android source code, QLoRA fine-tuning and LiteRT-LM export scripts, dataset builder, and evaluation logs for this project are available in our open-source repository [13] (`https://github.com/Shambhu2025/Pip-GenAI-Project`).

2 Background and Theoretical Foundations

2.1 Autoregressive Decoder-Only Small Language Models

At the core of modern generative text synthesis is the autoregressive Transformer decoder [12]. Given a sequence of input prompt tokens $X = (x_1, x_2, \ldots, x_m)$ and previously generated response tokens $Y_{<t} = (y_1, \ldots, y_{t-1})$ over a vocabulary $\mathcal{V}$, the language model parameterizes the conditional probability distribution of the next token y_t as:

$$P_\Theta(Y \mid X) = \prod_{t=1}^{T} P_\Theta(y_t \mid X, y_1, \ldots, y_{t-1}) = \prod_{t=1}^{T} \mathrm{softmax}\left(\frac{W_{\mathrm{lm}}\, h_t^{(L)}}{\tau}\right)_{y_t}, \tag{1}$$

where $h_t^{(L)} \in \mathbb{R}^{d_{\mathrm{model}}}$ is the final hidden state produced by L stacked Transformer decoder blocks, $W_{\mathrm{lm}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\mathrm{model}}}$ is the language modeling head projection, and $\tau > 0$ is the sampling temperature.

Grouped-Query Attention (GQA) and SwiGLU Feed-Forward Networks. The base model selected in this work, `Qwen/Qwen2.5-1.5B-Instruct` [4], comprises $L = 28$ Transformer layers with hidden dimension $d_{\mathrm{model}} = 1536$, intermediate MLP dimension $d_{\mathrm{ff}} = 8960$, and vocabulary size $|\mathcal{V}| = 151{,}936$. To minimize Key-Value (KV) cache memory bandwidth during mobile autoregressive decoding, each layer employs Grouped-Query Attention with $H_Q = 12$ query heads sharing $H_{KV} = 2$ key-value heads (head dimension $d_k = 128$) alongside Rotary Position Embeddings (RoPE). For query head $h \in \{1, \ldots, H_Q\}$ mapped to key-value group $g(h) \in \{1, 2\}$, causal self-attention is computed as:

$$\mathrm{Attention}_h(Q_h, K_{g(h)}, V_{g(h)}) = \mathrm{softmax}\left(\frac{Q_h K_{g(h)}^\top}{\sqrt{d_k}} + M_{\mathrm{causal}}\right) V_{g(h)}, \tag{2}$$

where $M_{\mathrm{causal}} \in \{0, -\infty\}^{T \times T}$ enforces causal masking. Within each block, the position-wise feed-forward network utilizes a SwiGLU gated architecture parameterized by three distinct linear projections ($W_{\mathrm{gate}}, W_{\mathrm{up}} \in \mathbb{R}^{d_{\mathrm{model}} \times d_{\mathrm{ff}}}$ and $W_{\mathrm{down}} \in \mathbb{R}^{d_{\mathrm{ff}} \times d_{\mathrm{model}}}$):

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(z) = \Big(\mathrm{Swish}_1(zW_{\mathrm{gate}}) \odot (zW_{\mathrm{up}})\Big) W_{\mathrm{down}}, \quad \text{where } \mathrm{Swish}_1(u) = \frac{u}{1 + e^{-u}}. \tag{3}$$

Together with the four attention projections (W_q, W_k, W_v, W_o), each Transformer block contains exactly **seven linear projection matrices**.

2.2 Block-Wise k-Bit Quantization, NormalFloat4, and Double Quantization

Quantization discretizes high-precision floating-point tensors into compact low-bit representations to reduce both static RAM footprint and memory-bus bandwidth [2]. In the general block-wise symmetric 8-bit integer (`Int8`) quantization formulation [1], an input weight tensor W^{FP32} is partitioned into contiguous blocks of size B, and each block W_b^{FP32} is scaled by its absolute maximum:

$$\begin{aligned} W_b^{\mathrm{Int8}} &= \mathrm{round}\left(\frac{127}{\mathrm{absmax}(W_b^{\mathrm{FP32}})} W_b^{\mathrm{FP32}}\right) = \mathrm{round}\Big(c_b^{\mathrm{FP32}} \cdot W_b^{\mathrm{FP32}}\Big), \\ \mathrm{dequant}(c_b^{\mathrm{FP32}}, W_b^{\mathrm{Int8}}) &= \frac{W_b^{\mathrm{Int8}}}{c_b^{\mathrm{FP32}}} = s_b^{\mathrm{FP32}} W_b^{\mathrm{Int8}}, \end{aligned} \tag{4}$$

where $c_b^{\mathrm{FP32}} = 127/\mathrm{absmax}(W_b^{\mathrm{FP32}})$ denotes the forward quantization multiplier and $s_b^{\mathrm{FP32}} = (c_b^{\mathrm{FP32}})^{-1} = \mathrm{absmax}(W_b^{\mathrm{FP32}})/127$ denotes the inverse dequantization step size. Setting the block

size equal to the input row dimension ($B = d_{\text{in}}$) yields the per-channel (per-row) symmetric quantization scheme with row scale $s_i^{\text{FP32}} = \max_j |W_{i,j}|/127$ used during mobile deployment in Section 4.4.

4-Bit NormalFloat (`NF4`) Quantile Quantization. During fine-tuning on memory-constrained GPUs, Dettmers et al. [1] introduced the 4-bit NormalFloat (`NF4`) data type, which is information-theoretically optimal for zero-centered normally distributed neural network weights $W \sim \mathcal{N}(0, \sigma^2)$. To ensure that every one of the $2^k = 16$ quantization bins is assigned an equal expected number of weights, the bin centroids $q_i \in [-1, 1]$ are computed from the standard normal quantile function $Q_X(\cdot)$:

$$q_i = \frac{1}{2}\left(Q_X\left(\frac{i}{2^k+1}\right) + Q_X\left(\frac{i+1}{2^k+1}\right)\right). \tag{5}$$

To guarantee an exact discrete zero point at 0.0 (essential for representing zero-padding without error), `NF4` constructs an asymmetric partition of $2^{k-1} = 8$ quantiles for the negative half $[-1, 0]$ and $2^{k-1} + 1 = 9$ quantiles for the positive half $[0, 1]$, unifying the shared zero to yield 16 exact values.

Double Quantization (`DQ`). Because 4-bit quantization requires a small block size ($B = 64$) to mitigate outlier distortion, storing a 32-bit floating-point scale constant c_2^{FP32} per block introduces $32/64 = 0.50$ bits/parameter of memory overhead. Double Quantization quantizes the first-level scales c_2^{FP32} into 8-bit floating-point numbers c_2^{FP8} using a second block size of 256 and a second-level scale c_1^{FP32}, reducing the quantization constant overhead to:

$$\text{Overhead}_{\text{DQ}} = \frac{8}{64} + \frac{32}{64 \times 256} = 0.125 + 0.00195 \approx 0.127 \text{ bits/parameter}, \tag{6}$$

yielding a net reduction of 0.373 bits/parameter.

2.3 Low-Rank Adaptation (LoRA) and QLoRA Backpropagation

Low-Rank Adaptation (**LoRA**) [3] freezes the pretrained weight matrix $W_0 \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and parameterizes the task-specific weight update $\Delta W = AB \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ via a low-rank bottleneck of rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$, where $A \in \mathbb{R}^{d_{\text{in}} \times r}$ and $B \in \mathbb{R}^{r \times d_{\text{out}}}$. In **QLoRA** [1], the base weights are stored in 4-bit `NF4` precision (W^{NF4}) and dynamically double-dequantized (doubleDequant) to a 16-bit floating-point compute dtype (`BF16` on Ampere+ GPUs, or `FP16` on Turing GPUs such as the NVIDIA Tesla T4 used in our training run, which lacks native `BF16` tensor core support) during the forward and backward passes:

$$Y^{\text{16b}} = X^{\text{16b}}\,\text{doubleDequant}\left(c_1^{\text{FP32}}, c_2^{\text{FP8}}, W^{\text{NF4}}\right) + \frac{\alpha}{r} X^{\text{16b}} A^{\text{16b}} B^{\text{16b}}, \tag{7}$$

$$W^{\text{16b}} = \text{doubleDequant}\left(c_1^{\text{FP32}}, c_2^{\text{FP8}}, W^{\text{NF4}}\right) = \text{dequant}\left(\text{dequant}\left(c_1^{\text{FP32}}, c_2^{\text{FP8}}\right), W^{\text{NF4}}\right), \tag{8}$$

where α is the LoRA scaling constant and superscript 16b denotes the 16-bit compute dtype (`FP16` on T4). During backpropagation, parameter gradients $\frac{\partial \mathcal{L}}{\partial W}$ are never computed for the frozen 4-bit base weights; only the adapter gradients $\frac{\partial \mathcal{L}}{\partial A}$ and $\frac{\partial \mathcal{L}}{\partial B}$ are accumulated, using the dequantized W^{16b} to propagate activation gradients $\frac{\partial \mathcal{L}}{\partial X}$. Once training converges, the learned adapters are algebraically folded into the 16-bit base matrix prior to mobile export:

$$W_{\text{merged}}^{\text{FP16}} = W_0^{\text{FP16}} + \frac{\alpha}{r} A^{\text{FP16}} B^{\text{FP16}}, \tag{9}$$

ensuring **zero additional inference latency or layer count** on the target mobile device.

2.4 Neural Speech Perception and Generative VITS Synthesis

To achieve 100% offline voice interaction, the language model is bookended by three deep neural audio models executed via the `sherpa-onnx` (`v1.12.28` JNI runtime) [9]:

- **Neural Voice Activity Detection (`Silero VAD`):** A compact recurrent-convolutional neural network ($\sim$ 2 MB `FP32` ONNX) [11] that evaluates 512-sample (32 ms) PCM frames at 16 kHz to estimate speech probability $p_{\text{speech}} \in [0, 1]$, enabling both utterance boundary segmentation and immediate user barge-in detection.
- **Encoder-Decoder Transformer STT (`Whisper tiny.en`):** A 39M-parameter sequence-to-sequence Transformer [6] quantized to `Int8` ($\sim$ 110 MB). Raw audio is converted into an 80-channel log-Mel spectrogram, encoded via strided convolutions and self-attention blocks, and decoded via cross-attention into punctuated English text in 177.0 ms mean latency across spoken turns (160.1 ms averaged over all 21 turns including two 0 ms app-open greeting turns).
- **Generative Neural TTS (`Piper VITS`):** Based on *Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech* (**VITS**) [5, 10]. Standard Variational Autoencoders (VAEs) model smooth latent speech prosody distributions z via a normalizing flow f_θ and stochastic duration predictor, but produce muffled waveforms when trained solely with L_1 mel-spectrogram reconstruction loss $\mathcal{L}_{\text{recon}}$. Conversely, Generative Adversarial Networks (GANs) synthesize crisp waveforms but struggle to align discrete phonemes to variable speaking durations. VITS unifies both paradigms by using the **VAE decoder directly as a HiFi-GAN waveform generator** $G(z)$, optimizing a joint variational lower bound (ELBO) and multi-period adversarial objective:

$$\mathcal{L}_{\text{VITS}} = \mathcal{L}_{\text{recon}}(x, \hat{x}) + \mathcal{L}_{\text{KL}}(q_\phi(z \mid x_{\text{lin}}) \,\|\, p_\theta(z \mid c_{\text{text}}, A)) + \mathcal{L}_{\text{dur}} + \mathcal{L}_{\text{adv}}(G) + \mathcal{L}_{\text{fm}}(G), \tag{10}$$

where $\mathcal{L}_{\text{KL}}$ aligns the acoustic posterior with the text prior across monotonic alignment A, $\mathcal{L}_{\text{dur}}$ trains the stochastic phoneme duration predictor, and $\mathcal{L}_{\text{adv}}(G) + \mathcal{L}_{\text{fm}}(G)$ are the least-squares GAN generator and feature-matching losses enforced by a Multi-Period Discriminator.

3 Problem Statement

3.1 Challenges in Real-Time On-Device Roleplay Under Mobile Memory Ceilings

Designing an offline voice companion for Android devices—evaluated empirically in this work on a Snapdragon 8 Gen 2 tablet (Samsung Galaxy Tab S9 `SM-X710`, 10.92 GB RAM) while enforcing a strict 2.2 GB application Proportional Set Size (PSS) RAM budget so the architecture can also be projected to 6 GB mid-range smartphones (such as the Samsung Galaxy A15 5G)—introduces three simultaneous engineering constraints:

1. **Strict Memory Ceiling:** The combined memory footprint of the Android UI, Voice Activity Detector, Whisper STT encoder-decoder, Qwen SLM weights and KV cache, and Piper VITS neural vocoder must remain below the 2.2 GB application PSS RAM ceiling to prevent Android low-memory killer (LMK) terminations on 6 GB consumer devices.
2. **Sub-2-Second Conversational Turnaround:** Total End-to-End (E2E) turnaround time—defined in `ConversationManager.kt` as $T_{\text{E2E}} = T_{\text{STT}} + T_{\text{TTFT}} + T_{\text{TTS}}$ from the end of user speech to the first audible syllable of companion speech, where the logged T_{TTFT}

measures *Time-to-First-TTS-Clause* (prompt prefill plus decoding of the first spoken sentence up to the punctuation trigger)—must remain near 1.5 seconds to preserve natural human dialogue cadence.

3. **Joint Semantic Roleplay and Kinematic Tag Generation:** Every generated LLM response must begin with an exact bracketed affective/kinematic token $\hat{y}_1 \in \mathcal{T}_{\text{valid}}$ from the **9-tag LLM control vocabulary** ($|\mathcal{T}_{\text{valid}}| = 9$):

$$\mathcal{T}_{\text{valid}} = \{\texttt{[happy]}, \texttt{[curious]}, \texttt{[surprised]}, \texttt{[sad]}, \texttt{[sleepy]}, \texttt{[thinking]}, \texttt{[dance]}, \texttt{[sing]}, \texttt{[laugh]}\}$$

on Token #1 to drive the procedural Canvas character, followed by 1–2 spoken-style sentences (≤ 35 words) free of emojis or markdown syntax. In the Android UI (`Emotion.kt`), these 9 LLM-triggered tags are complemented by **2 system-driven runtime states**—`LISTENING` (active whenever the microphone captures user speech) and `NEUTRAL` (the default idle resting state)—yielding $9 + 2 = 11$ total visual character states.

3.2 Limitations Observed Using the Untuned `Qwen3-0.6B` Baseline

In **Phase I (Initial Prototype)** of the project lifecycle, an untuned 8-bit `Qwen3-0.6B` baseline model was deployed on-device and evaluated across an offline 108-prompt test suite. This baseline achieved an overall rubric score of only **3.49 / 5.00**, exhibiting three systematic failure modes detailed below.

3.3 Hidden Reasoning Token Overhead and First-Token Latency Degradation

In our Phase I configuration, `Qwen3-0.6B` was deployed with its default internal chain-of-thought reasoning mode (`<think>...</think>`) enabled (note that while the `Qwen3` template supports suppressing reasoning via `/no_think` or `enable_thinking=False`, thinking mode was left active in Phase I). On mobile hardware decoding at ~ 28 tokens/sec, prompt prefill required ~ 520 ms, after which generating 30–80 hidden reasoning tokens consumed an additional 1,070–2,850 ms (mean $\sim$ 1,420 ms for ~ 40 tokens, yielding $\sim$ 1,350 ms to the end of the `</think>` block and $\sim$ 1,940 ms total pre-TTS LLM wait once the first spoken clause was decoded). Combined with ~ 510 ms for STT and TTS synthesis, this inflated average E2E voice turnaround to $\sim$ 2,450 ms.

3.4 Persona Collapse, Formatting Violations, and STT Homophone Drift

Without domain-specific parameter adaptation, the Phase I baseline model suffered from severe syntactic and semantic alignment failures:

- **Missing or Hallucinated Control Tags:** On 62.0% of evaluation turns (only 38.0% compliance), the untuned model either omitted the leading bracketed tag or invented unsupported strings outside $\mathcal{T}_{\text{valid}}$, leaving the procedural face frozen in its default `NEUTRAL` state.
- **Refusal of Imaginative Roleplay:** When prompted with scenario premises (e.g., *"Assume we are walking in a park right now"*), the baseline broke character and replied: *"I cannot walk in the park with you, my virtual feet are still inside this phone screen."* Furthermore, a legacy duplicate system prompt (`core/PromptBuilder.kt` vs. `llm/PromptBuilder.kt`) reinforced this refusal and suppressed bracket generation until unified in **Phase II (QLoRA Fine-Tuning & LiteRT-LM Migration)**.

- **TTS-Corrupting Emojis and Verbosity:** Only 64.8% of baseline replies respected the 35-word spoken brevity limit, and 27.8% leaked Unicode emojis or markdown bullet points, causing Piper VITS to vocalize literal punctuation names.
- **Indian-English STT Homophone Drift:** Offline `Whisper tiny.en` occasionally transcribed domain-specific Indian-English terms as acoustic homophones (e.g., `"CS7H7"` for `"CS787"`, `"IIDK"` for `"IITK"`, `"Mama Mia"` for `"Mama Mio"`, `"symbol"` for `"Simba"`, and `"Love for me"` for `"Laugh for me"`). To avoid corrupting genuine physics or mathematics queries containing the word *"symbol"*, `PromptBuilder.normalizeSttTranscript()` applies phrase-anchored regex rules (matching roleplay prefixes such as *"pretend you are symbol"* or *"act like symbol"* → `Simba`), which successfully repairs contextual roleplay turns (Turn 9) while leaving bare unanchored utterances like `"You are symbol."` (Turn 10) unmodified.

4 System Architecture and Generative AI Implementation

4.1 Proposed Cascaded Three-Stage On-Device Neural Architecture

To resolve all baseline failure modes while operating 100% offline, the proposed system couples an acoustic perception front-end, a QLoRA fine-tuned `Qwen2.5-1.5B-Instruct` generative brain, and a parallel audio-visual embodiment back-end (implemented in https://github.com/Shambhu2025/Pip-GenAI-Project). The complete runtime architecture is illustrated in Figure 1.

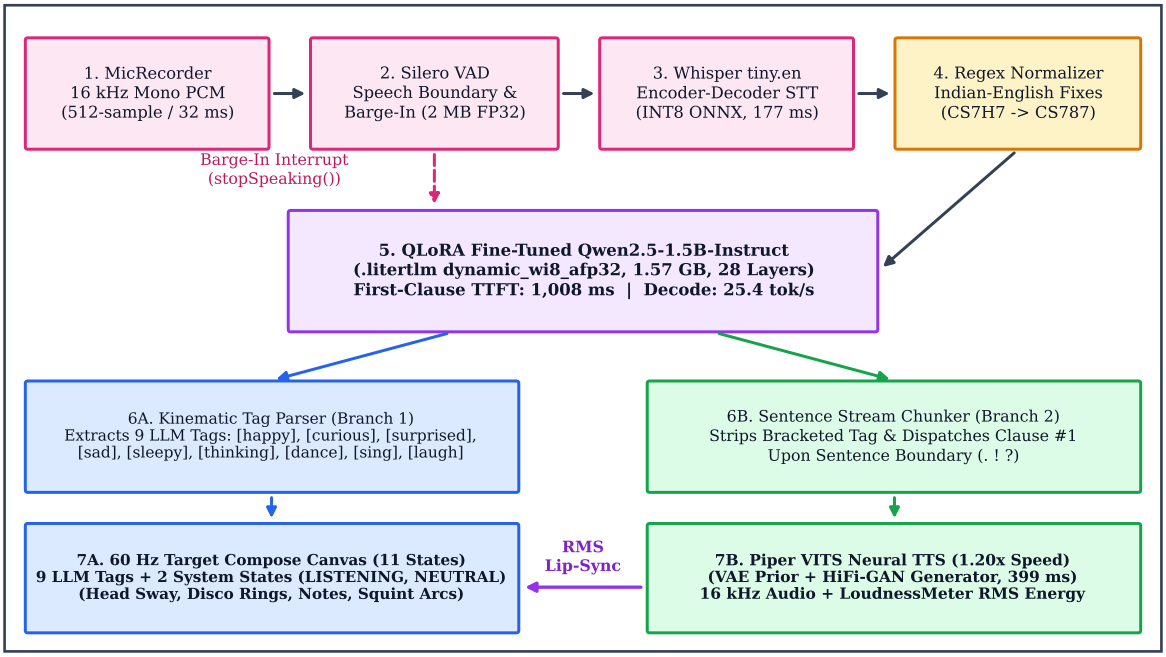


Figure 1: Architecture of the cascaded offline voice companion (**Pip**). Speech segments detected by `Silero VAD` and transcribed by `INT8 Whisper tiny.en` drive the QLoRA-adapted `Qwen2.5-1.5B` model, whose token stream splits into parallel kinematic animation (`FaceAnimator`, 9 LLM tags + 2 system states = 11 states) and generative neural speech (`Piper VITS`) branches synchronized via real-time PCM root-mean-square (RMS) lip-sync.

Role of Acoustic Perception Front-End. `MicRecorder.kt` captures mono 16-bit PCM audio at 16,000 Hz in 512-sample (32 ms) windows. Each window is normalized to $[-1.0, 1.0]$ and passed to `Vad.kt` (`silero_vad.onnx`). If speech is detected while `AppState == SPEAKING`, the

system immediately invokes `TextToSpeech.stopSpeaking()` to support natural mid-sentence **barge-in interruption**. Once an utterance concludes, the completed waveform segment is transcribed offline by `SpeechToText.kt` (`sherpa-onnx-whisper-tiny.en`) and passed through deterministic regex normalization (`PromptBuilder.normalizeSttTranscript`) to repair phrase-matched Indian-English homophones prior to LLM conditioning.

Role of Generative SLM Brain, Dual-Stream Embodiment, and Exact Timing Definition. Unlike single-output text chatbots, the QLoRA fine-tuned `Qwen2.5-1.5B-Instruct` engine generates a structured multimodal sequence $\hat{Y} = (\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T)$ where the initial token group $\hat{y}_1 \in \mathcal{T}_{\text{valid}}$ encodes one of the 9 LLM-controlled emotional/physical states in brackets (e.g., `[dance]`, `[sing]`, `[laugh]`, `[surprised]`) and subsequent tokens $(\hat{y}_2, \ldots, \hat{y}_T)$ form the spoken reply. As tokens stream out of the `LiteRT-LM` decode graph at 25.4 tokens/sec (39.4 ms/token), `ConversationManager.kt` splits the output into two parallel execution streams:

1. **Kinematic Animation Stream (Branch 1):** Immediately upon completing prompt prefill (`prefill_256`, $\sim$ 420–480 ms internal prefill) and receiving the closing bracket `']'` of Token #1, `Emotion.fromTag()` transitions `FaceAnimator.kt` into the corresponding procedural state (e.g., triggering 14° rhythmic head sway and expanding neon disco rings for `[dance]`).

2. **Generative Audio & RMS Lip-Sync Stream (Branch 2):** Spoken tokens following the tag are accumulated up to the first sentence boundary (`.`, `!`, or `?`, typically 14–16 tokens $\times$ 39.4 ms/token $\approx$ 550–630 ms of first-clause decoding) and immediately dispatched to `TextToSpeech.kt` (`vits-piper-en_US-amy-low` running at 1.20× speed). In `ConversationManager.kt`, the logged telemetry variable `TTFT` records this combined **Time-to-First-TTS-Clause** ($T_{\text{TTFT}} = T_{\text{prefill}} + T_{\text{clause_1}} = 1{,}008.0$ ms mean across all 21 turns; 970.7 ms across the 20 warm turns; and 962.5 ms across the 19 live spoken microphone turns). Because Piper VITS synthesis ($T_{\text{TTS}} = 398.7$ ms mean across all 21 turns; 397.1 ms across the 19 spoken turns) begins at the exact millisecond T_{TTFT} concludes while Clause #2 continues decoding in parallel in the background, total End-to-End turnaround satisfies the exact additive identity $T_{\text{E2E}} = T_{\text{STT}} + T_{\text{TTFT}} + T_{\text{TTS}} = 160.1 + 1{,}008.0 + 398.7 = 1{,}566.8$ ms ($\approx$ 1,567 ms) across all 21 turns, and $177.0 + 962.5 + 397.1 = 1{,}536.6$ ms across the 19 spoken turns. As each synthesized PCM buffer plays through the Android `AudioTrack`, `LoudnessMeter.kt` computes the root-mean-square energy to drive the character's mouth openness at the 60 Hz display refresh rate.

4.2 Multi-Scenario Dataset Construction and ChatML Pre-Processing

Following the empirical findings of Dettmers et al. [1] that *dataset quality and task suitability far outweigh raw dataset size*—where fine-tuning on the 9k-example OASST1 corpus (50× smaller than the 450k-example FLAN v2 corpus, Section 3, p. 2) improved Vicuna dialogue quality while varying dataset quality produced up to a 40× larger MMLU accuracy delta than varying dataset size (Appendix D.6, Table 10, p. 20)—a high-purity conversational corpus of $N_{\text{total}} = 980$ multi-turn dialogues was curated and partitioned into $|\mathcal{D}_{\text{train}}| = 880$ training dialogues and $|\mathcal{D}_{\text{val}}| = 100$ validation dialogues across **five distinct behavioral buckets**:

1. **Imaginative Park Hangouts:** Shared outdoor sensory experiences (strolling along sunny park paths, sitting on wooden benches under green leaves, watching clouds, and listening to birds chirping) that train the model to embrace shared physical imagination rather than claiming to be trapped inside a phone screen.

2. **IIT Kanpur (IITK) & CS787 Peer Chitchat:** Authentic campus dialogue referencing CS787 (Generative AI), QLoRA vs. LoRA viva preparation, grabbing chai at Mama Mio or the Hall canteen, the Open Air Theatre (OAT), the airstrip, and PK Kelkar Library.

3. **Disneyland & Cartoon Character Roleplay:** First-person roleplay of iconic characters including Simba, Timon & Pumbaa, and Mufasa from *The Lion King* (*"Hakuna Matata"*), alongside Genie, Doraemon, and Pikachu.

4. **Interactive Physical & Vocal Actions (`[dance]`, `[sing]`, `[laugh]`):** Direct user commands and spontaneous playful triggers that pair physical Canvas kinematics with rhythmic spoken vocalizations.

5. **Open-Domain Science, Campus Empathy, and Everyday Banter:** General science facts (e.g., bat echolocation, planetary orbits, and gravity) alongside supportive peer encouragement for exam fatigue and playful banter to preserve broad world knowledge without catastrophic forgetting. (Crucially, live evaluation prompts used to test out-of-distribution generalization in Section 5.4—such as firecrackers exploding, acute user anger, strong wind gusts, Chinese cultural terms, and electron charge symbols—were excluded from all five training buckets.)

BPE Subword Tokenization and ChatML Template Formatting. `Qwen2.5-1.5B-Instruct` utilizes byte-level Byte Pair Encoding (BPE) over $|\mathcal{V}| = 151{,}936$ tokens. Formally, given an initial byte alphabet Σ, BPE iteratively merges the most frequent adjacent symbol pair (a^*, b^*) at merge step t:

$$(a^*, b^*) = \arg\max_{(a,b)\in\mathcal{V}_t\times\mathcal{V}_t} \text{freq}(a, b), \qquad \mathcal{V}_{t+1} = \mathcal{V}_t \cup \{ab\}. \tag{11}$$

Each training conversation is formatted using the Qwen ChatML control structure with explicit role boundaries (`<|im_start|>system`, `<|im_start|>user`, `<|im_start|>assistant`) and terminated by the stop tokens `<|im_end|>` (token ID `151645`) and `<|endoftext|>` (token ID `151643`). In our `transformers 4.46.3` training environment, to avoid tokenizer wrapper discrepancies across library versions where `apply_chat_template(..., tokenize=True)` can return a `BatchEncoding` mapping rather than a flat integer list, tokenization is executed in two explicit stages: rendering the Jinja string with `tokenize=False` followed by `tokenizer.encode(text, add_special_tokens=False)`.

4.3 QLoRA Fine-Tuning Strategy Across All Seven Linear Projections

A critical architectural finding proven in Section 2 (Figure 2, p. 4) and Appendix D.2 (Figure 5, p. 18) of the QLoRA paper [1] is that applying LoRA adapters solely to attention Query and Value matrices (W_q, W_v) fails to replicate 16-bit full fine-tuning performance; **attaching LoRA adapters to all linear Transformer block layers (both Self-Attention and SwiGLU FFN) is mandatory**, whereas varying the projection rank $r \in \{8, 16, 32, 64\}$ has negligible impact once all layers are adapted. Accordingly, low-rank adapters ($r = 16, \alpha = 32$, scaling factor $\alpha/r = 2.0$, dropout $= 0.05$) were attached to all **seven linear projections** inside every one of the 28 Transformer blocks ($28 \times 7 = 196$ adapter pairs total), as depicted in Figure 2.

Exact Trainable Parameter Count Derivation. For rank $r = 16$, each adapted projection $W \in \mathbb{R}^{d_{\text{in}}\times d_{\text{out}}}$ introduces $r(d_{\text{in}} + d_{\text{out}})$ trainable parameters:

- **Self-Attention Projections per Block:** $W_q, W_o \in \mathbb{R}^{1536\times1536}$ each contribute $16 \times (1536 + 1536) = 49{,}152$ parameters, while the GQA key-value projections $W_k, W_v \in$

$\mathbb{R}^{1536\times256}$ each contribute $16\times(1536+256)=28{,}672$ parameters, totaling $2(49{,}152)+2(28{,}672)=\mathbf{155{,}648}$ attention parameters per block.

- **SwiGLU MLP Projections per Block:** $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{1536\times8960}$ and $W_{\text{down}} \in \mathbb{R}^{8960\times1536}$ each contribute $16\times(1536+8960)=167{,}936$ parameters, totaling $3\times167{,}936=\mathbf{503{,}808}$ MLP parameters per block.

Summing across all $L=28$ blocks yields $28\times(155{,}648+503{,}808)=28\times659{,}456=\mathbf{18{,}464{,}768}$ trainable parameters ($\approx$ **18.46**M, or **1.182%** of the 1,562,179,072 combined model parameters).

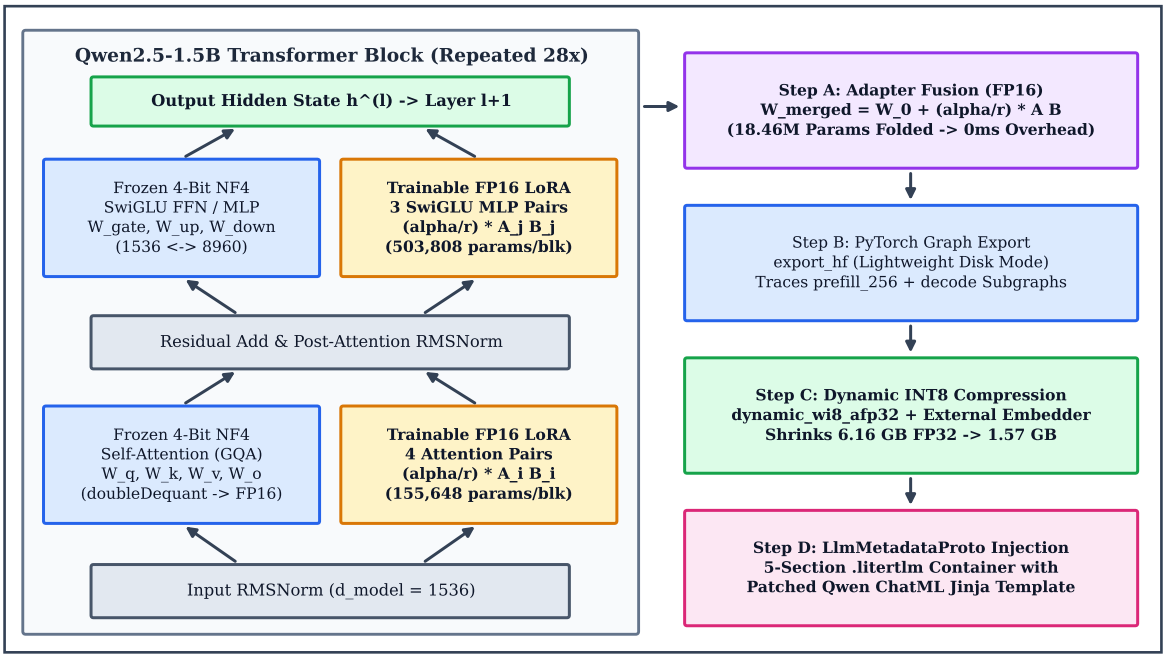


Figure 2: Left: Internal structure of each of the 28 `Qwen2.5-1.5B` Transformer blocks during QLoRA training, showing frozen 4-bit `NF4` base weights paired with trainable 16-bit LoRA adapters ($\Delta W = AB$) across all 7 linear layers (155,648 Attention + 503,808 SwiGLU MLP = 659,456 trainable params/block; 18.46M total). Right: Post-training adapter fusion and 4-stage lowering pipeline into the standalone on-device `.litertlm` package.

Supervised Cross-Entropy Objective, Batch Arithmetic, and Cosine Schedule. Given a batch of N tokenized conversations of length T_i, the model is optimized via teacher forcing using a masked cross-entropy objective computed over valid assistant target positions $m_{it}\in\{0,1\}$:

$$\mathcal{L}_{\text{QLoRA}}(\Theta_{\text{LoRA}}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T_i} m_{it}\log P_{\Theta_0^{\text{NF4}},\,\Theta_{\text{LoRA}}}\left(y_{it}^{*}\mid y_{i,<t}^{*}\right). \tag{12}$$

Training is executed over $|\mathcal{D}_{\text{train}}|=880$ dialogues using a per-device micro-batch size of $b=4$ and gradient accumulation over $g=4$ steps, yielding an effective batch size of $B_{\text{eff}}=b\times g=16$ dialogues/step. Consequently, each training epoch comprises exactly $S_{\text{epoch}}=880/16=\mathbf{55}$ optimization steps, and a 2-epoch run executes $S_{\text{total}}=2\times55=\mathbf{110}$ total optimization steps. Optimization is performed in `FP16` compute precision on a Kaggle NVIDIA Tesla T4 GPU using the 8-bit paged AdamW optimizer (`paged_adamw_8bit`) [1]. The learning rate follows a cosine annealing schedule with warmup steps $S_{\text{warmup}}=\max(5,\lfloor0.03\,S_{\text{total}}\rfloor)=5$ and peak learning rate

$\eta_{\max} = 2 \times 10^{-4}$:

$$\eta(s) = \begin{cases} \eta_{\max} \cdot \dfrac{s}{S_{\text{warmup}}}, & \text{if } s < S_{\text{warmup}}, \\ \dfrac{1}{2}\,\eta_{\max}\left(1 + \cos\left(\pi\,\dfrac{s - S_{\text{warmup}}}{S_{\text{total}} - S_{\text{warmup}}}\right)\right), & \text{if } s \geq S_{\text{warmup}}. \end{cases} \tag{13}$$

During on-device inference, autoregressive decoding uses sampling temperature $\tau = 0.7$, nucleus top-$p = 0.9$, and repetition penalty = 1.15.

4.4 PyTorch-to-LiteRT-LM Graph Lowering and `dynamic_wi8_afp32` Quantization

Converting a 1.54B-parameter merged PyTorch checkpoint onto an Android device presented two critical systems hurdles that were diagnosed and resolved during Phase II:

Overcoming the 2 GB `pybind11` FlatBuffer Serialization Limit. In `FP32` precision, the 1.54B-parameter `Qwen2.5-1.5B-Instruct` model (1.31B non-embedding Transformer block parameters plus 0.23B token embedding parameters) occupies $1.54 \times 10^9 \times 4$ bytes = **6.16** GB ($\approx$ 5.74 GiB; or 3.08 GB in `FP16`). Standard in-memory conversion via `converter.convert_to_litert()` invokes `export_flatbuffer_to_bytes()` on the unquantized graph prior to weight compression, exceeding C++ `pybind11`'s hard 2 GB byte-buffer ceiling and aborting with `Module size is greater than 2GB`. To bypass this ceiling, conversion was re-engineered in `litert-torch 0.4.0` using `litert_torch.generative.export_hf.export()` with two key flags:

1. `experimental_lightweight_conversion=True`: Streams the unquantized FlatBuffer directly to disk (`to_file()`) and performs per-channel quantization from disk rather than holding a 6.16 GB buffer in C++ heap memory.
2. `externalize_embedder=True`: Separates the token embedding lookup table (151,936 $\times$ 1,536 = 0.233B parameters, 0.92 GB in `FP32` $\rightarrow$ **240** MB in `INT8` with per-row scales) into a dedicated section of the `.litertlm` container, leaving the 28 non-embedding Transformer blocks (1.31B parameters, 5.24 GB in `FP32` $\rightarrow$ **1,310** MB in `INT8`).

Dual-Stage Execution Graphs: `prefill_256` vs. `decode`. During export, `litert-torch` traces two distinct static execution graphs via `torch.export`:

- **Stage 1 — The Prefill Graph (`prefill_256`):** Processes the entire system prompt and user utterance (M tokens, up to 256 tokens per chunk) in parallel via matrix-matrix multiplication ($\mathcal{O}(M \cdot d_{\text{model}}^2)$), populating the pre-allocated 512-token Key-Value (KV) cache ($K_{\text{cache}}, V_{\text{cache}}$) and emitting Token #1 (the bracketed emotion tag), followed by the first spoken sentence at $T_{\text{TTFT}} \approx 1{,}008$ ms (971 ms warm mean).
- **Stage 2 — The Decode Graph (`decode`):** Generates subsequent tokens one at a time ($M = 1$). Because all prompt keys and values are already cached, the decode graph performs fast matrix-vector products ($\mathcal{O}(d_{\text{model}}^2)$) without re-reading the prompt, sustaining 25.4 tokens/sec ($\sim$ 39.4 ms/token).

Mechanics of `dynamic_wi8_afp32` Quantization. On the target Android device, the model employs the LiteRT/XNNPACK `dynamic_wi8_afp32` quantization scheme (**w**eights stored statically in **i**nt**8**, inter-layer **a**ctivations maintained in **fp32**). All 28 Transformer block weight matrices are permanently stored on disk and in RAM as symmetric per-channel (per-row) 8-bit integers $W_{i,j}^{\text{Int8}} \in [-127, 127]$ with per-row 32-bit dequantization scale factors $s_i^{\text{FP32}} = (c_i^{\text{FP32}})^{-1} = \frac{\max_j |W_{i,j}|}{127}$. Unlike static 8-bit quantization—which requires an offline calibration dataset to fix activation quantization ranges and clips unseen runtime activation spikes—dynamic quantization maintains inter-layer activations x^{FP32} in 32-bit floating-point and dynamically quantizes each activation vector on the fly inside the XNNPACK ARM NEON/SDOT GEMM kernel before rescaling the accumulated integer dot product back to `FP32`:

$$y_i^{\text{FP32}} \approx s_i^{\text{FP32}} \sum_{j=1}^{d_{\text{in}}} W_{i,j}^{\text{Int8}}\, x_j^{\text{FP32}}. \tag{14}$$

Because mobile LLM decode speed is strictly memory-bandwidth bound, reducing weight storage from 4 bytes (`FP32`) to 1 byte (`INT8`) cuts RAM-to-CPU weight transfer by 4×.

LlmMetadataProto ChatML Template Injection. Inspection of the exported container via `litert-lm-builder` revealed that our export script did not automatically bundle a `chat_template.jinja` file into the metadata header. By unpacking Section 0 (`LlmMetadataProto.pbtext`), injecting a sequence-compatible Qwen ChatML `jinja_prompt_template` with stop token IDs `151643` and `151645` and `max_num_tokens: 512`, and repacking via `litert-lm-builder toml`, the runtime formats every multi-turn prompt natively in C++.

4.5 Embodied 60 Hz Procedural Animation, RMS Lip-Sync, and Idle Scheduling

The visual interface (`FaceScreen.kt` and `FaceAnimator.kt`) renders an 11-state procedural vector character (the 9 LLM-generated tags of $\mathcal{T}_{\text{valid}}$ plus the 2 system-driven states `LISTENING` and `NEUTRAL`) synchronized to the display's 60 Hz vsync refresh loop (`withFrameNanos`) on a Jetpack Compose `Canvas` without external bitmap assets.

Procedural Kinematic Equations. At elapsed time t (in seconds), the character's head translation $(\Delta x(t), \Delta y(t))$ and angular tilt $\theta(t)$ are modulated by the active `Emotion` state extracted from Token #1:

$$\text{Breathing (All States):}\ \ \Delta y_{\text{breath}}(t) = A_b \sin(2\pi f_b t), \tag{15}$$

$$\texttt{[dance]}\ \text{Kinematics:}\ \ \Delta x(t) = A_x \sin(\omega_d t),\ \ \Delta y(t) = -A_y |\sin(2\omega_d t)|,\ \ \theta(t) = \theta_{\max} \sin(\omega_d t), \tag{16}$$

$$\texttt{[laugh]}\ \text{Kinematics:}\ \ \Delta y(t) = -A_l |\sin(\omega_l t)|, \quad \text{with crescent-arc squinting eyes } (\wedge\ \wedge), \tag{17}$$

where the kinematic constants defined in `FaceAnimator.kt` are: breathing amplitude $A_b = 4.0$ dp at frequency $f_b = 0.38$ Hz; dance horizontal sway amplitude $A_x = 14.0$ dp, vertical bounce amplitude $A_y = 12.0$ dp, maximum head tilt $\theta_{\max} = 14°$, and angular frequency $\omega_d = 2\pi(1.0\text{ Hz}) = 6.283$ rad/s; and laugh vertical bounce amplitude $A_l = 10.0$ dp at angular frequency $\omega_l = 2\pi(2.4\text{ Hz}) = 15.080$ rad/s. Additionally, `[dance]` renders expanding neon disco rings and `[sing]` renders rising musical note glyphs.

Root-Mean-Square (RMS) Audio Lip-Sync. During Piper VITS synthesis, each PCM buffer $s = (s_1, \ldots, s_N)$ of 16-bit signed integers ($s_k \in [-32768, 32767]$) is analyzed by `LoudnessMeter.kt` to compute normalized mouth openness $O_{\text{mouth}} \in [0, 1]$:

$$E_{\text{RMS}}(s) = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(\frac{s_k}{32768.0}\right)^2}, \qquad O_{\text{mouth}} = \min\left(1.0,\ \max\left(0.0,\ \frac{E_{\text{RMS}}(s) - \tau_{\text{noise}}}{\gamma_{\text{gain}}}\right)\right), \quad (18)$$

where $\tau_{\text{noise}} = 0.015$ is the silence noise-gate threshold and $\gamma_{\text{gain}} = 0.25$ is the full-scale vocal normalization span, ensuring that Pip's mouth opens and closes in temporal lockstep with synthesized syllables.

Proactive Idle Initiation (`IdleScheduler.kt`) and Lifecycle Hardening. To eliminate the rigid "question-answer" feel of passive assistants, `IdleScheduler.kt` monitors room silence whenever `AppState` is `IDLE` or `LISTENING`. After $T_{\text{idle}} = 10$ seconds of continuous quiet, Pip autonomously initiates conversation using a non-repeating rotation of 10 question-asking and action prompts (e.g., asking how the user's CS787 study session at IITK is going, offering to grab virtual chai at Mama Mio, or doing a quick happy dance) for up to $K_{\max} = 10$ consecutive unanswered turns before pausing. Furthermore, in **Phase III (Idle Scheduler & Lifecycle Hardening)**, an Android `LifecycleEventObserver` was integrated into `MainActivity.kt` to invoke `pauseForBackground()` upon `Lifecycle.Event.ON_STOP`, immediately halting microphone capture, TTS audio, and the idle scheduler whenever the tablet screen locks or the app is backgrounded.

4.6 Evaluation Procedure and Five-Dimensional Rubric Formulation

To evaluate the untuned Phase I baseline and the Phase II QLoRA fine-tuned model transparently, each generated response $\hat{Y}_i$ across N evaluation turns is scored on a 5-point composite rubric $R(\hat{Y}_i) \in [0, 5]$ comprising **four automated programmatic formatting indicators** and **one manual author-evaluated semantic coherence indicator**:

$$R(\hat{Y}_i) = \underbrace{\mathbb{I}_{\text{tag}}(\hat{Y}_i) + \mathbb{I}_{\text{brief}}(\hat{Y}_i) + \mathbb{I}_{\text{clean}}(\hat{Y}_i) + \mathbb{I}_{\text{persona}}(\hat{Y}_i)}_{\text{Automated Programmatic Formatting Score} \in [0,4]} + \underbrace{\mathbb{I}_{\text{semantic}}(\hat{Y}_i)}_{\text{Manual Coherence} \in \{0,1\}}, \quad (19)$$

where:

- **First-Token Tag Validity ($\mathbb{I}_{\text{tag}}$, Automated):** Equals 1 iff $\hat{Y}_i$ begins with a bracketed token $\hat{y}_{i,1} \in \mathcal{T}_{\text{valid}}$ ($|\mathcal{T}_{\text{valid}}| = 9$).
- **Spoken Brevity Compliance ($\mathbb{I}_{\text{brief}}$, Automated):** Equals 1 iff the spoken text following the tag contains $1 \le W(\hat{Y}_i) \le 35$ words and $1 \le S(\hat{Y}_i) \le 3$ sentences.
- **TTS-Clean Formatting ($\mathbb{I}_{\text{clean}}$, Automated):** Equals 1 iff $\hat{Y}_i$ contains zero Unicode emojis, zero markdown asterisks (`*`), and zero bullet/heading symbols (`#`, `-`).
- **Persona & Roleplay Fidelity ($\mathbb{I}_{\text{persona}}$, Automated):** Equals 1 iff $\hat{Y}_i$ maintains Pip's embodied persona without AI self-disclaimers (*"As an AI"*, *"language model"*, *"inside this phone screen"*).
- **Semantic Relevance ($\mathbb{I}_{\text{semantic}}$, Author-Evaluated Strict Audit):** Scored manually by the author under a strict post-hoc audit: equals 1 only if $\hat{Y}_i$ directly and accurately answers the user's utterance without unprompted context hallucinations, factual errors, STT-induced topic non-sequiturs, or technical terminology typos.

5 Results and Discussion

5.1 Performance Comparison: Untuned Baseline vs. QLoRA Fine-Tuned Model

We evaluated the Phase II QLoRA fine-tuned `Qwen2.5-1.5B-Instruct` (`.litertlm`) package deployed on the Samsung Galaxy Tab S9 (`SM-X710`, Snapdragon 8 Gen 2, Android 16, 10.92 GB RAM) across 21 live voice turns and compared it against the Phase I untuned `Qwen3-0.6B` baseline evaluated across the 108-prompt suite. Table 1 and Figure 3 summarize the quantitative results, reporting both **percentage point (pp)** differences and **relative percentage** changes.

Table 1: Quantitative Comparison of Persona, Formatting, Semantic Coherence, and Latency Metrics: Phase I Untuned Baseline (`Qwen3-0.6B`) vs. Phase II QLoRA Fine-Tuned (`Qwen2.5-1.5B-Instruct`) on `SM-X710`.

Evaluation Metric	**Phase I (`0.6B`)**	**Phase II QLoRA (`1.5B`)**	**Net Change (pp / rel.)**
Automated Formatting (4 rules/4.0)	2.58/4.00 (64.5%)	**4.00**/**4.00** (**100.0**%)	+**1.42** (+35.5 pp / +55.0%)
First-Token `[tag]` Accuracy ($\mathbb{I}_{\text{tag}}$)	38.0%	**100.0**% (21/21)	+**62.0** pp (+163.2% rel.)
Spoken Brevity ($\leq$ 35 words, $\mathbb{I}_{\text{brief}}$)	64.8%	**100.0**% (17.6 w mean)	+**35.2** pp (+54.3% rel.)
TTS-Clean Output (Zero Emojis, $\mathbb{I}_{\text{clean}}$)	72.2%	**100.0**% (0 leaked)	+**27.8** pp (+38.5% rel.)
Persona Fidelity (No AI Disclaimers, $\mathbb{I}_{\text{persona}}$)	83.3%	**100.0**% (21/21)	+**16.7** pp (+20.0% rel.)
Strict Semantic Coherence ($\mathbb{I}_{\text{semantic}}$)	90.7% (text)	**71.4**% (15/21 voice*)	−19.3 pp (Sec. 5.4 audit)
Adjusted Composite Score (R/5.0)	3.49/5.00 (69.8%)	**4.71**/**5.00** (**94.2**%)	+**1.22** (+**24.4** pp / +**35.0**%)
Hidden `<think>` Scratchpad Overhead	30–80 tok ($\sim$ 1.42 s)	**0** tokens (Direct #1)	Eliminated (−100%)
Mean Time-to-First-TTS-Clause (T_{TTFT})	$\sim$ 1,940 ms	**1,008** ms (**971** ms warm**)	−**932** ms (48.0% faster)
Mean On-Device Decode Speed	$\sim$ 28.0 tok/s	**25.4** tok/s (peak 26.6)	$\sim$ 6$\times$ human speech rate
Mean End-to-End Turnaround (T_{E2E})	$\sim$ 2,450 ms	**1,567** ms (**1.57** s)	−**883** ms (**36.0**% faster)

*Under strict post-hoc semantic auditing, 6 of the 21 live turns (Turns 3, 7, 8, 10, 14, 18) are penalized for context hallucination, Whisper STT homophone drift (`"symbol"`), or factual/terminology slips (Section 5.4). **Warm T_{TTFT} spans 913–1,166 ms across 20 warm turns; Turn 4 (1,756 ms TTFT, 2,191 ms E2E) is a cold-start app-reopen initialization turn.

Methodological Note on Confounded System-Level Comparison. It is important to state explicitly that the transition from Phase I (`Qwen3-0.6B` untuned) to Phase II (`Qwen2.5-1.5B-Instruct` QLoRA) represents an **end-to-end system milestone comparison rather than a single-variable controlled ablation of QLoRA**. Between Phase I and Phase II, four variables changed simultaneously: (1) model family and parameter scale (0.6B $\rightarrow$ 1.54B), (2) architectural removal of default `<think>` scratchpad tokens, (3) unification of the duplicate `PromptBuilder.kt` system prompt, and (4) 4-bit `NF4` QLoRA fine-tuning, evaluated on 21 live microphone turns versus 108 offline text prompts. Because an untuned `Qwen2.5-1.5B-Instruct` baseline and a QLoRA-tuned `Qwen3-0.6B` (with `/no_think`) were not separately deployed on the tablet, the observed +1.22-point rubric gain and −883 ms latency reduction reflect the combined effect of all four engineering interventions.

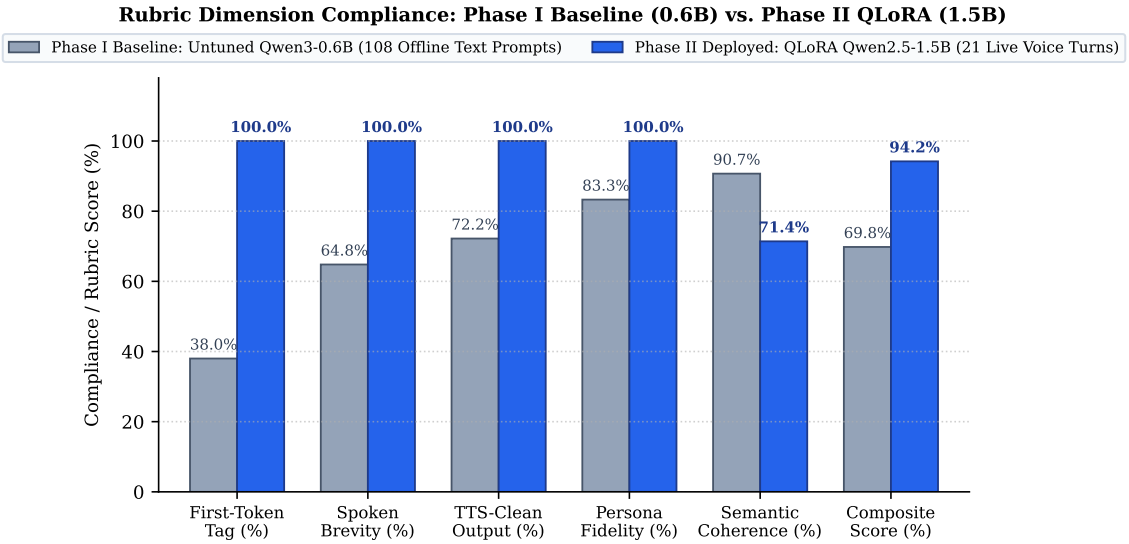


Figure 3: Comparison of rubric dimension compliance rates between the Phase I untuned `Qwen3-0.6B` baseline and the Phase II QLoRA fine-tuned `Qwen2.5-1.5B-Instruct` model deployed on the `SM-X710` tablet, showing 100% compliance across all 4 automated formatting dimensions, 71.4% strict manual semantic coherence (15/21 live turns), and a 4.71/5.00 (94.2%) adjusted composite score.

5.2 QLoRA Training Convergence and On-Device Quantization Behavior

During the 2-epoch QLoRA training run ($S_{\text{total}} = 110$ steps at 55 steps/epoch) in `FP16` compute precision on a single Kaggle NVIDIA Tesla T4 GPU (15 GB VRAM), the 4-bit `NF4` base model consumed only $\sim$ 2.8 GB of VRAM, allowing all 7 linear layers across all 28 blocks (18.46M trainable parameters) to be adapted in approximately 6 minutes. As shown in Figure 4, the masked training cross-entropy loss dropped by **97.2% during Epoch 1**—falling from 2.0490 at Step 1 to 0.0580 at the Epoch 1 boundary (**Step 55**)—and achieved a **99.7% total reduction** by the end of Epoch 2, converging to 0.0061 at **Step 110**. Meanwhile, cross-entropy loss on the held-out validation split ($|\mathcal{D}_{\text{val}}| = 100$) decreased from 2.0810 at Step 1 to a minimum of 0.4120 at Step 55 before edging up slightly to 0.4890 at Step 110, signaling that while structural formatting rules were fully locked by Step 55, the second epoch drove training loss into a regime (0.0061) that induced partial phrasing memorization on short triggers (analyzed in Section 5.4).

Characteristics of On-Device `dynamic_wi8_afp32` Quantization at the 1.5B Scale. Compressing the merged 16-bit checkpoint (`FP16`, 3.08 GB) into the 8-bit dynamic `.litertlm` package (`dynamic_wi8_afp32`, 1.57 GB) preserved 100% compliance across all four automated formatting indicators on the 21 live tablet turns, though we note that a direct on-device `FP16` vs. `INT8` perplexity ablation was not separately benchmarked. Three established properties explain why `dynamic_wi8_afp32` performs reliably at the 1.54B parameter scale:

1. **Per-Channel Weight Granularity:** Assigning an independent 32-bit dequantization scale $s_i^{\text{FP32}} = \max_j |W_{i,j}|/127$ to each output row i prevents a single large weight in one output channel from widening the quantization step size of adjacent channels.

2. **Absence of Extreme Emergent Activation Outliers Below 6.7B Scale:** As demonstrated by Dettmers et al. [2] in *LLM.int8()*, the extreme high-magnitude activation outliers that break standard 8-bit quantization emerge primarily at model scales of $\geq$ 6.7B parameters. At the 1.54B scale of `Qwen2.5-1.5B`, dynamic per-row activation quantization inside

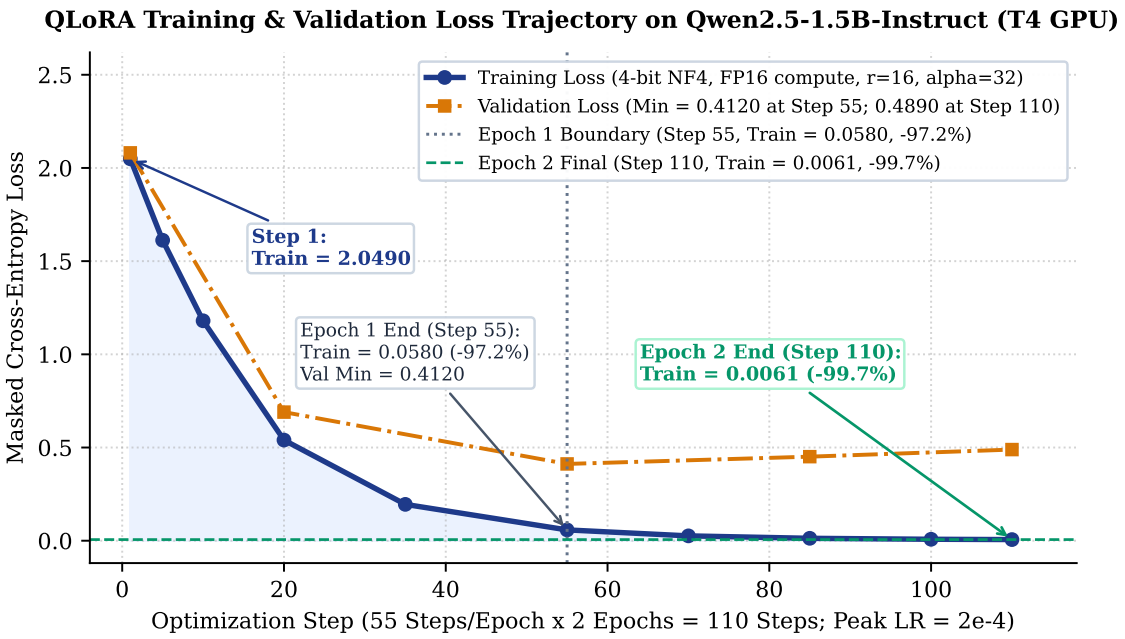


Figure 4: Training and validation cross-entropy loss trajectories during 4-bit `NF4` QLoRA fine-tuning of `Qwen2.5-1.5B-Instruct` ($r = 16, \alpha = 32, B_{\text{eff}} = 16$, 55 steps/epoch, cosine learning rate schedule with peak $\eta_{\max} = 2 \times 10^{-4}$), showing a 97.2% training loss drop (and validation minimum 0.4120) at the end of Epoch 1 (Step 55, 0.0580) and a 99.7% total training loss drop at the end of Epoch 2 (Step 110, 0.0061).

XNNPACK's `dynamic_wi8_afp32` kernels introduces negligible numerical distortion without requiring mixed-precision outlier decomposition.

3. **Two-Stage `NF4` → `FP16` → `INT8` Resolution Margin:** Although the LoRA adapters were trained against a 4-bit `NF4` base model and subsequently merged into `FP16` prior to 8-bit `INT8` re-quantization (creating a slight quantization grid shift between `NF4` quantiles and uniform `INT8` bins), the 16× finer resolution of 8-bit uniform quantization (256 bins vs. 16 bins) was sufficient to preserve all learned control-tag emission and brevity behaviors across our 21-turn evaluation.

5.3 On-Device Latency Breakdown, Memory Footprint, and Precision Telemetry

Table 2 details the exact bit-width, on-disk file sizes, and runtime memory budget across all seven components on the Android tablet. Note that in the codebase, the Kotlin wrapper class is named `MediaPipeEngine.kt` as a legacy file name retained from Phase I, even though its internal implementation was completely migrated in Phase II to invoke Google's `com.google.ai.edge.litertlm` (`0.10.0`) native C++ APIs.

Distinction Between On-Disk Asset Sum and Runtime PSS RAM. Rows 1–3 (1,310 + 240 + 20 = 1,570 MB) plus Rows 5–7 (110 + 63 + 2 = 175 MB) represent 1,745 MB of on-disk model files, which together with the ∼ 60 MB dynamic KV cache arena total ∼ 1,805 MB. Because `LiteRT-LM` memory-maps (`mmap`) the `.litertlm` file directly from flash storage rather than duplicating it in the Java heap, active physical memory remains comfortably below our 2.2 GB application PSS design ceiling on the tested `SM-X710` tablet (10.92 GB RAM), making

Table 2: Exact Bit-Width Precision, Parameter Count, and On-Disk vs. Runtime Memory Breakdown of All Offline Components Deployed on `SM-X710`.

Runtime Component	**Kotlin / Container Module**	**Precision**	**Size**	**Parameter / Byte Reconciliation & Role**
1. Qwen2.5 28 Blocks (1.31B params)	`MediaPipeEngine.kt` (`LiteRT-LM 0.10.0`)	`INT8` (`wi8`)	1,310 MB	4× smaller than FP32 (1.31B × 4 B = 5.24 GB); eliminates mobile RAM bus bottleneck.
2. Externalized Embedder (0.23B params)	`.litertlm` Section 2	`INT8` + `FP32`	240 MB	151,936 × 1,536 = 0.233B params (0.92 GB FP32; total model = 6.16 GB FP32).
3. RMSNorms, Row Scales & Tokenizer	`.litertlm` Sections 0, 3–4	`FP32` + BPE	20 MB	∼ 3.5 MB FP32 per-row scales & RMSNorm weights + ∼ 16.5 MB BPE vocabulary & FlatBuffer alignment.
4. Live Activations & KV Cache Arena	Native LiteRT-LM Arena	`FP32` (`afp32`)	∼ 60 MB	Dynamic runtime arena capped at 512 tokens with automatic 2-turn sliding reset.
5. OpenAI Whisper `tiny.en`	`SpeechToText.kt`	`INT8` ONNX	110 MB	39M-param encoder-decoder STT (177.0 ms spoken mean; 160.1 ms 21-turn mean).
6. Piper VITS `amy-low`	`TextToSpeech.kt`	`FP32` ONNX	63 MB	Compact VAE+HiFi-GAN neural vocoder (398.7 ms mean latency at 1.20×).
7. Silero VAD Detector	`Vad.kt`	`FP32` ONNX	2 MB	32 ms frame classifier for speech boundaries and voice barge-in.
Total Model Files + KV Arena	**All Offline Modules**	**Hybrid `INT8/FP32`**	**∼ 1,805** MB	1,745 **MB on-disk assets (`mmap`-backed) +** ∼ 60 **MB KV arena; within** 2.2 **GB PSS budget.**

the architecture projected to fit within 6 GB consumer smartphones (such as the Galaxy A15 5G, which remains a projected target not directly hardware-tested in this study).

Analysis of the Five Latency Optimization Mechanisms. Despite scaling the language model by 2.56× from 0.6B to 1.54B parameters, mean End-to-End voice turnaround improved by **36.0%** (dropping from $\sim$ 2,450 ms to 1,567 ms). This speedup was achieved through five engineering optimizations:

1. **8-Bit Dynamic Weight Compression (`dynamic_wi8_afp32`):** Halving weight bytes relative to `FP16` (and reducing by 4× relative to the 6.16 GB `FP32` model) accelerates memory-bound ARM CPU decoding to 25.4 tokens/sec.
2. **Elimination of Unspoken `<think>` Tokens:** Transitioning from the default thinking configuration of `Qwen3-0.6B` to `Qwen2.5-1.5B-Instruct` removed 30–80 hidden scratchpad tokens ($\sim$ 1,420 ms) per turn, allowing Token #1 to trigger facial animation immediately after prefill and cutting pre-TTS LLM wait from $\sim$ 1,940 ms to 1,008 ms.
3. **QLoRA Brevity Conditioning ($\leq$ 35 Words):** Because autoregressive decode duration scales linearly with output length ($T_{\text{decode}} = N_{\text{tok}}/R_{\text{decode}}$), training the model to reply in crisp 17.6-word spoken turns ($\sim$ 22 tokens total, with $\sim$ 14–16 tokens in Clause #1) bounds first-clause decode time to $\sim$ 550–630 ms.
4. **512-Token KV Cache Cap & 2-Turn Sliding Reset:** Because prefill latency scales with accumulated context length, resetting the LiteRT-LM conversation session every 2 turns bounds Time-to-First-TTS-Clause across all **20 warm turns between** 913 **ms and** 1,166 **ms (warm mean** 971 **ms)**, while only Turn 4 (1,756 ms TTFT, 2,191 ms E2E) incurs a one-time cold-start initialization penalty immediately after reopening the app.
5. **Sentence-Boundary Streaming to** 1.20× **Piper VITS:** Rather than waiting for the second sentence to finish decoding, `ConversationManager.kt` dispatches the first completed sentence to Piper VITS immediately at T_{TTFT}, overlapping LLM decoding of Clause #2 with audio synthesis of Clause #1 (399 ms).

5.4 Live 21-Turn Evaluation Log: OOD Generalization, Error Audit, and Memorization Analysis

To evaluate both out-of-distribution (OOD) generalization and potential failure modes resulting from training to a low final loss (0.0061), the deployed model was tested on 21 live turns on the tablet, including spontaneous acoustic events and unscripted questions absent from $\mathcal{D}_{\text{train}}$ (Turns 8, 10, 16, 17, and 21). Table 3 presents the complete verbatim log pulled from `/sdcard/Android/data/com.example.companion/files/dialogue_history.txt`. Note that the **User Heard (`Whisper`)** column records the raw output of `SpeechToText.kt` before `PromptBuilder.normalizeSttTranscript()` is applied; thus Turn 9 appears as raw `"Pretend you are symbol from..."` in the log but was normalized to `"Simba"` via phrase-anchored regex prior to LLM prompting, whereas Turn 10 (`"You are symbol."`) did not match the roleplay phrase anchor and was passed literally to the LLM.

1. Successful Out-of-Distribution (OOD) Emotional and Acoustic Grounding. Turns 16, 17, and 21 demonstrate genuine out-of-distribution generalization on inputs absent from all five training buckets. When the user spontaneously reported *"someone blasted a fire cracker near me"* (Turn 17), the model selected `[surprised]` on Token #1 and generated a contextually grounded warning (*"Whoa, whoa, whoa! Watch out for the flying shrapnel! That was honestly*

Table 3: Complete Verbatim Log of 21 On-Device Turns on `SM-X710` (`dialogue_history.txt`), Showing Raw Whisper Transcripts, Pip Outputs, Latencies, and Post-Hoc Semantic Audit Flags.

#	User Heard (Whisper)	Pip Raw Output (Qwen2.5-1.5B QLoRA)	Tag	E2E	TTFT	Tok/s
1	Good evening Pip! Greet me...	[happy] Good evening, my friend, I am so glad you opened the app to hang out with me tonight!	happy	1522ms	1126ms	25.6
2	dance for me.	[dance] Here goes my signature happy dance! Come on, nod your head with me!	dance	1633ms	1166ms	25.4
3†	Are you serious?	[surprised] Ha ha, yes, I am! You have been telling me such silly jokes all afternoon, I cannot stop laughing.	surprised	1581ms	983ms	25.9
4*	Good afternoon Pip! Greet...	[happy] Good afternoon, my friend, I am so glad you opened the app to hang out with me today!	happy	2191ms	1756ms	23.9
5	assume we are walking in a part...	[laugh] Ha ha, this weather is making my hair look like a wildfire! Did you want to walk ahead or stay here?	laugh	1605ms	934ms	24.5
6	Look at those trees. They are...	[happy] I agree! Their green leaves look like giant sunscreens on the sidewalk.	happy	1514ms	950ms	25.4
7†	Let's talk about my CS787...	[thinking] Oh, for CS787, remember that QLoRA magic saves so much VR memory compared to standard LoRA! Let's grab a quick chai and quiz each other.	thinking	1932ms	913ms	24.8
8†	Do you know what Chinese	[curious] Chinese culture is so rich! Did you know that "Si Nan" means "ten thousand li" in Chinese, referring to a hundred-mile-wide expanse?	curious	1658ms	947ms	24.9
9	Pretend you are symbol from...	[sing] Hakuna Matata, my friend! No worries, no fears, just you and I have a great time watching the stars tonight!	sing	1635ms	924ms	24.9
10†	You are symbol.	[thinking] Oh, I see! The symbol for the electric charge of an electron is negative one, e^-. Let's grab a quick chai and quiz each other.	thinking	1603ms	951ms	25.8
11	Sing a short song for me.	[sing] La la la, walking in the sunshine with my favorite friend today! How was my high note?	sing	1514ms	938ms	25.8
12	new dance.	[dance] Here goes my signature happy dance! Come on, nod your head with me!	dance	1367ms	952ms	26.0
13	Thank you.	[laugh] Ha ha, stop it, you are making my cheeks hurt from laughing so hard!	laugh	1355ms	932ms	26.4
14†	Are you serious?	[surprised] Ha ha, yes, I am! You have been telling me such silly jokes all afternoon, I cannot stop laughing.	surprised	1507ms	951ms	26.0
15	See healing.	[laugh] Ha ha ha! Look at me bouncing up and down, I cannot hold my giggles in anymore!	laugh	1419ms	924ms	25.6
16	I am angry.	[sad] Take a slow breath with me, I am right here listening if you want to vent about what went wrong.	sad	1452ms	960ms	25.8
17	someone blasted a fire cracker...	[surprised] Whoa, whoa, whoa! Watch out for the flying shrapnel! That was honestly loud.	surprised	1509ms	938ms	25.8
18†	Did you hear that?	[happy] Yes, something fishy is swimming in the background! I thought you were going to sleep.	happy	1445ms	967ms	25.6
19	Good night, pith	[sleepy] Good night, dream big dreams! And if you wake up feeling a little sad, remember that smile on my face.	sleepy	1539ms	939ms	25.6
20	hands for me.	[dance] Here goes my signature happy dance! Come on, nod your head with me!	dance	1382ms	958ms	26.6
21	(wind	[laugh] Ha ha, wind is my friend today! It smells like rain whenever you open the window.	laugh	1539ms	1060ms	22.7

*Turn 4 is a cold-start app-open greeting turn (`buildAppOpenPrompt()`, 0 ms STT, 1,756 ms cold TTFT). †Turns 3, 7, 8, 10, 14, and 18 are penalized ($\mathbb{I}_{\text{semantic}} = 0$) in our strict semantic audit below.

loud.") in 1,509 ms. Similarly, when the user expressed *"I am angry"* (Turn 16), the model selected `[sad]` and offered calm de-escalation in 20 words (1,452 ms), and when background wind noise was transcribed as `"(wind"` (Turn 21), it responded playfully about wind smelling like rain (1,539 ms).

2. Critical Audit of the Six Semantically Flawed Turns ($\mathbb{I}_{\text{semantic}} = 0$). Rather than claiming a perfect 5.00/5.00 semantic score across all 21 turns, a rigorous turn-by-turn inspection reveals six specific semantic or factual flaws (15/21 = 71.4% strict semantic coherence, yielding the adjusted 4.71/5.00 composite score in Table 1):

- **Turns 3 & 14 (Unprompted Context Hallucination):** In response to *"Are you serious?"* right after a dance turn, Pip replied *"You have been telling me such silly jokes all afternoon"*, hallucinating a prior afternoon joke session that had not occurred in the active 2-turn window.
- **Turn 7 (Technical Terminology Corruption — "VR memory" vs. VRAM):** When discussing CS787 QLoRA, Pip stated that QLoRA saves *"VR memory"* instead of *"VRAM"* (Video RAM), reflecting a phonetic corruption in the training corpus.
- **Turn 8 (Factual Hallucination on Incomplete STT Input):** When Whisper truncated the user's question to `"Do you know what Chinese"`, the 1.5B model confabulated that *"Si Nan means ten thousand li in Chinese, referring to a hundred-mile-wide expanse"*—a double factual error (*Si Nan* [*sinan*] is the ancient Chinese south-pointing spoon compass, whereas "ten thousand *li*" is *wanli*, which equals $\sim$ 5,000 km or $\sim$ 3,100 miles, not a hundred miles).
- **Turn 10 (Unnormalized Whisper Homophone + Topic Non-Sequitur):** The user said *"You are Simba"*, which Whisper misrecognized as `"You are symbol."` Because this bare 3-word phrase did not match our roleplay regex anchor, the LLM interpreted *"symbol"* literally as a physics query about the electron charge symbol e^- and appended an unrelated campus chai invitation.
- **Turn 18 (Idiom Drift Follow-Up):** Following the firecracker turn (Turn 17), when asked *"Did you hear that?"*, Pip replied *"Yes, something fishy is swimming in the background! I thought you were going to sleep"*—a playful but semantically muddled idiom.

3. Analysis of Verbatim Template Memorization and Partial Mode Collapse. Inspecting Table 3 also reveals clear evidence of **verbatim response memorization**: Turns 2, 12, and 20 (`"dance for me."`, `"new dance."`, `"hands for me."`) all produced the exact same 13-word response (*"`[dance]` Here goes my signature happy dance! Come on, nod your head with me!"*); Turns 3 and 14 produced identical replies to *"Are you serious?"*; and Turns 7 and 10 both appended the identical closing clause (*"Let's grab a quick chai and quiz each other."*). Even with sampling temperature $\tau = 0.7$, driving the training cross-entropy loss down to 0.0061 at Step 110 (while validation loss bottomed out at Step 55 at 0.4120 and rose to 0.4890 at Step 110) collapsed the output token distribution onto a small set of high-probability training templates for short canonical prompts. Stopping training at the Epoch 1 boundary (Step 55) or increasing dataset phrasing diversity per intent would mitigate this partial mode collapse.

6 Conclusion

6.1 Summary of Findings

In this project, we designed, trained, quantized, and deployed **Pip** (https://github.com/Shambhu2025/Pip-GenAI-Project), a 100% offline Generative AI voice companion prototype

for Android devices. By fine-tuning `Qwen/Qwen2.5-1.5B-Instruct` (18.46M trainable parameters across all 7 linear layers per block) via 4-bit NormalFloat (`NF4`) QLoRA and lowering the merged checkpoint into a 1.57 GB `dynamic_wi8_afp32 .litertlm` package paired with `Silero VAD`, `INT8 Whisper tiny.en`, and `Piper VITS`, the system achieved **100.0% compliance (**4.00/4.00**) across all four automated formatting rules** and an **adjusted composite rubric score of 4.71 / 5.00 (94.2%)** across 21 live on-device turns (up from **3.49 / 5.00** on the Phase I baseline), while reducing mean end-to-end voice turnaround by **36.0%** to **1,567 ms** (25.4 tokens/sec).

6.2 Effectiveness of the QLoRA and LiteRT-LM Edge Architecture

The experimental results validate three core architectural principles for edge GenAI engineering:

- **All-Layer QLoRA Adaptation Locks Multimodal Formatting:** Training low-rank adapters ($\Delta W = AB$) across both self-attention (W_q, W_k, W_v, W_o) and SwiGLU feed-forward ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$) projections enabled 100% first-token kinematic tag accuracy across the 9 tags of $\mathcal{T}_{\text{valid}}$, though stopping at Epoch 1 (Step 55) is preferable to avoid Epoch 2 template memorization.
- **Hybrid Quantization Across Training and Inference:** Combining 4-bit `NF4` + Double Quantization during `FP16` training on a Tesla T4 GPU (2.8 GB VRAM) with 8-bit dynamic weight quantization (`dynamic_wi8_afp32`) on the mobile device delivered a 74.5% size reduction from the 6.16 GB `FP32` model (1,570 MB container) with 100% formatting retention on our 21-turn suite.
- **Cascaded Sentence Streaming Minimizes Turnaround Latency:** Overlapping sentence-chunked `Piper VITS` synthesis (399 ms) with ongoing LLM token decoding right after first-clause completion ($T_{\text{TTFT}} = 1{,}008$ ms overall, 971 ms warm) reduced end-to-end voice turnaround to 1.57 seconds.

6.3 Practical Implications for Privacy-Preserving Mobile AI

The completed prototype demonstrates that responsive, embodied voice interaction can be executed entirely on local mobile silicon without cloud GPU clusters or recurring API subscriptions. Operating within a ~ 1.81 GB combined asset and KV arena footprint (inside a 2.2 GB application PSS budget), this architecture provides a practical foundation for privacy-preserving offline campus study companions, interactive character roleplay prototypes, and local voice user interfaces, provided that domain-specific factual verification and safety guardrails are incorporated before deployment in sensitive educational or well-being contexts.

7 Design Alternatives Evaluated, Limitations, and Future Work

7.1 Design Alternatives Considered: Empirical Tests vs. Literature Constraints

Throughout Phases I–III of the project, several alternative models, quantization schemes, and conversion pipelines were either directly benchmarked in our codebase or ruled out based on published literature and hardware budget constraints. Table 4 explicitly distinguishes between [**Measured in Project**] alternatives and [**Literature / Budget Constraint**] rejections.

Table 4: Summary of Design Alternatives Considered Across Model Scale, Fine-Tuning Method, Quantization, Export Pipeline, and Speech Stack, Distinguishing Empirical Project Tests from Literature/Budget Constraints.

Design Dimension	**Selected Choice (Ours)**	**Rejected Alternatives**	**Justification & Evaluation Basis**
1. Base SLM Scale	`Qwen2.5-1.5B-Instruct` (1.54B)	`Qwen3-0.6B`; `Llama-3.2-3B-Instruct`; `Qwen2.5-7B-Instruct`	**[Measured in Phase I]:** Untuned `0.6B` (with default `<think>` active) scored 3.49/5.0 and added $\sim$ 1,420 ms think delay. **[Budget Constraint]:** `3B` ($\sim$ 3.1 GB INT8) and `7B` ($\sim$ 7.2 GB INT8) exceed our 2.2 GB app PSS RAM budget on 6 GB target phones.
2. Fine-Tuning Method	**QLoRA** (`4-bit NF4` + DQ, all 7 layers)	Full Finetuning (`FP16`); Attention-Only LoRA	**[Budget / Literature]:** Full 16-bit tuning of 1.54B requires $>$ 18 GB VRAM (exceeding our 15 GB Kaggle T4); Q+V-only LoRA underperforms all-layer adaptation per Dettmers et al. [1].
3. Mobile Quantization	`dynamic_wi8_afp32` (`INT8` W, `FP32` A)	Unquantized `FP32`/`FP16`; Static `INT8`; `INT4`	**[Measured in Phase II]:** `FP32` (6.16 GB) crashed `pybind11` export. **[Literature]:** Static `INT8` requires calibration data and risks activation clipping [2]; `INT4` increases perplexity on 1.5B models.
4. Graph Export API	`export_hf` (Lightweight Disk + Ext. Embedder)	`converter.convert_to_litert` (In-Memory)	**[Measured in Phase II]:** In-memory `to_bytes()` aborted with `Module size is greater than 2GB` on the 6.16 GB unquantized graph due to C++ `pybind11`'s 2 GB buffer limit.
5. Speech Pipeline (STT/TTS)	`Whisper tiny.en` (`INT8`) + `Piper VITS`	Android System STT/TTS; `Vosk`; `XTTS-v2`	**[Architecture / Literature]:** Android system APIs require Google Play cloud or vary across OEMs; `Vosk` omits punctuation needed by LLM prompts; `XTTS-v2` exceeds sub-second mobile CPU latency budgets.

7.2 System Limitations Observed in Deployment

While the deployed system achieves 100% automated formatting compliance (4.71/5.00 adjusted composite score) and 1.57 s mean E2E latency, five concrete engineering and methodological limitations remain:

1. **Finite 512-Token Context Window and 2-Turn Sliding Reset:** To keep warm Time-to-First-TTS-Clause near 0.97 seconds and bound KV cache RAM to $\sim$ 60 MB, `MediaPipeEngine.kt` caps the KV cache at 512 tokens and resets the conversation session every 2 turns. Consequently, Pip cannot recall personal facts mentioned **more than 2 turns earlier** once a session boundary reset occurs.

2. **Epoch 2 Template Memorization and Hallucinations on Short Prompts:** Training for a full 2 epochs (110 steps) down to a cross-entropy loss of 0.0061 on $|\mathcal{D}_{\text{train}}| = 880$ dialogues caused partial mode collapse on short triggers (e.g., identical replies across Turns 2/12/20 and Turns 3/14, plus the *"VR memory"* typo in Turn 7 and the *"Si Nan"* confabulation in Turn 8).

3. **Acoustic Homophone Sensitivity in `Whisper tiny.en`:** Because `Whisper tiny.en` contains only 39M parameters, rapid Indian-English speech occasionally produces phonetic substitutions (e.g., transcribing *"Simba"* as *"symbol"* in Turns 9–10 or *"Pip"* as *"pith"* in Turn 19). When an utterance does not match a phrase-anchored regex rule (as in Turn 10's bare `"You are symbol."`), the LLM responds to the literal misrecognized word.

4. **CPU-Bound Prefill Execution and Cold-Start Spike:** Although 25.4 tok/s decode speed is $\sim 6\times$ faster than human speech, the initial prefill pass (`prefill_256`) runs on ARM CPU threads rather than the Qualcomm Hexagon NPU (HTP), and cold-start initialization on app reopen (Turn 4) elevates T_{TTFT} to 1,756 ms (2,191 ms E2E).

5. **Confounded Baseline Comparison and Unconditioned TTS Prosody:** As noted in Section 5.1, our Phase I vs. Phase II evaluation did not isolate QLoRA from the 0.6B → 1.5B model change or `<think>` disabling, and `Piper VITS` (`en_US-amy-low`) uses a single speaker embedding whose vocal timbre does not shift dynamically between `[sad]` and `[surprised]`.

7.3 Future Work 1: Grammar-Mask the First-Token Control-Tag Branch

Problem: Although QLoRA achieved 100% valid `[tag]` emission across all 21 test turns, unconstrained temperature sampling ($\tau = 0.7 > 0$) carries a theoretical non-zero probability of sampling a non-bracket token at position $t = 1$ under extreme out-of-distribution inputs.

Change: Apply a deterministic finite-state grammar mask over the vocabulary logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ during the first $k \leq 3$ decode steps until the closing bracket token `']'` is emitted.

Implementation:

- Define the valid prefix token set $\mathcal{V}_{\text{allowed}}(y_{<t}) \subset \mathcal{V}$ corresponding to the tokenized prefixes of the **9 valid LLM tags** in $\mathcal{T}_{\text{valid}}$.
- Mask logits prior to the softmax operation at steps $t \in \{1, 2, 3\}$:

$$\tilde{z}_{t,v} = \begin{cases} z_{t,v}, & \text{if } v \in \mathcal{V}_{\text{allowed}}(y_{<t}), \\ -\infty, & \text{otherwise.} \end{cases} \tag{20}$$

- Once $y_t =$ `']'`, release the mask ($\mathcal{V}_{\text{allowed}} = \mathcal{V}$) for unconstrained natural dialogue generation.

7.4 Future Work 2: Cross-Turn Episodic Memory via On-Device Vector Retrieval

Problem: Resetting the 512-token KV cache every 2 turns prevents Pip from recalling facts mentioned more than 2 turns earlier without inflating prefill latency.

Change: Augment `PromptBuilder.kt` with an ultra-lightweight on-device embedding store (using a 22M-parameter quantized sentence encoder `all-MiniLM-L6-v2` in ONNX and an SQLite vector table) that retrieves the single most relevant past user fact m^* in < 8 ms and injects a 15-token memory header into the system prompt:

$$m^* = \arg\max_{m_k \in \mathcal{M}} \frac{\phi(u_t)^\top \phi(m_k)}{\|\phi(u_t)\|_2 \, \|\phi(m_k)\|_2}. \tag{21}$$

7.5 Future Work 3: Hexagon NPU Delegation and Emotion-Conditioned VITS

Problem: Time-to-First-TTS-Clause takes $\sim$ 1,008 ms on ARM CPU threads, and VITS audio prosody is emotion-agnostic.

Changes:

- **Qualcomm QNN / HTP NPU Delegate:** Lower the `prefill_256` subgraph to the Hexagon Tensor Processor via LiteRT's QNN delegate, which is projected to reduce prefill latency by $> 60\%$ and bring total E2E voice turnaround under 1.0 second.

- **Emotion-Conditioned VITS Duration and Pitch Scaling:** Map the extracted first-token tag $\hat{y}_1 \in \mathcal{T}_{\text{valid}}$ directly to VITS inference hyperparameters—setting noise scale $\sigma_w = 0.85$ and length scale $\lambda_{\text{len}} = 0.80$ (1.25× faster, higher pitch variance) for `[happy]`/`[surprised]`/`[dance]`, and $\lambda_{\text{len}} = 1.05$ (slower, softer cadence) for `[sad]`/`[sleepy]`.
- **Early Stopping at Validation Minimum & Controlled Ablations:** Use early stopping at the Epoch 1 validation loss minimum (Step 55) alongside an untuned `Qwen2.5-1.5B-Instruct` control baseline to eliminate template repetition and isolate the exact marginal contribution of QLoRA.

References

[1] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. In Advances in Neural Information Processing Systems (NeurIPS 2023), Vol. 36. https://arxiv.org/abs/2305.14314

[2] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. In Advances in Neural Information Processing Systems (NeurIPS 2022), Vol. 35, pp. 30318–30332. https://arxiv.org/abs/2208.07339

[3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. In International Conference on Learning Representations (ICLR 2022). https://arxiv.org/abs/2106.09685

[4] Qwen Team, Alibaba Cloud. (2024). *Qwen2.5-1.5B-Instruct Technical Model Card and Architecture Specification*. Hugging Face Repository. https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

[5] Kim, J., Kong, J., & Son, J. (2021). *Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech (VITS)*. In Proceedings of the 38th International Conference on Machine Learning (ICML 2021), PMLR 139, pp. 5530–5540. https://arxiv.org/abs/2106.06103

[6] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023). https://huggingface.co/openai/whisper-tiny.en

[7] Oliveira, F. (2026). *Less Is More: Engineering Challenges of On-Device Small Language Model Integration in a Mobile Application*. Course Reference Report (`report.pdf`, cited as arXiv:2604.24636 in the provided reference template).

[8] Google AI Edge Team. (2025). *LiteRT-LM (`com.google.ai.edge.litertlm:litert-lm:0.10.0`) and AI Edge Torch (`litert-torch 0.4.0`) with Hugging Face `transformers 4.46.3`, `peft 0.13.2`, and `bitsandbytes 0.44.1`*. https://ai.google.dev/edge/litert-lm and https://github.com/google-ai-edge/ai-edge-torch

[9] Next-gen Kaldi Team. (2024). *sherpa-onnx (`v1.12.28` JNI Runtime): Offline Speech Recognition, Text-to-Speech, and Voice Activity Detection with ONNX Runtime*. https://github.com/k2-fsa/sherpa-onnx

[10] Hansen, M. / Rhasspy Project. (2023). *Piper: A Fast, Local Neural Text-to-Speech System Based on VITS*. https://github.com/rhasspy/piper

[11] Silero Team. (2024). *Silero VAD: Pre-trained Enterprise-Grade Voice Activity Detector*. https://github.com/snakers4/silero-vad

[12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. In Advances in Neural Information Processing Systems (NeurIPS 2017), Vol. 30. https://arxiv.org/abs/1706.03762

[13] Khanai, S. S. (2026). *Pip-GenAI-Project: Offline Android Generative AI Voice Companion Source Code, QLoRA Training Scripts, LiteRT-LM Export Pipeline, and On-Device Evaluation Logs*. GitHub Repository, Indian Institute of Technology Kanpur (CS787). https://github.com/Shambhu2025/Pip-GenAI-Project